FinApp – Master Dokumentacja (Wersja Alpha / MVP)

Projekt: FinApp – Osobisty Asystent Finansowy (ZPI)
Status: Wersja Alpha (MVP zintegrowane End-to-End)
Stos technologiczny: * **Frontend:** Flutter (Dart), Riverpod, Dio, fl_chart

- **Backend:** Python, FastAPI, SQLAlchemy, SQLite (z opcją migracji na PostgreSQL)
Ostatnia aktualizacja: Maj/Czerwiec 2026

---

1. Podsumowanie Projektu (Executive Summary)

FinApp to wieloplatformowa aplikacja mobilna do kompleksowego zarządzania finansami osobistymi. Architektura systemu opiera się na wydajnym backendzie napisanym w FastAPI oraz dynamicznym frontendzie we Flutterze.

Głównym celem wersji MVP było zrealizowanie historii użytkowników (User Stories) związanych z agregacją majątku: oszczędności, kont emerytalnych, inwestycji giełdowych i kryptowalut, a także prognozowaniem emerytury.

Decyzje architektoniczne i cięcia (Zakres MVP): Ze względów czasowych i w celu dowiezienia stabilnej wersji Alpha, podjęto świadomą decyzję o **wycięciu funkcji Update (Edycji)**. Aplikacja skupia się na solidnym wdrożeniu operacji **Create, Read, Delete (CRD)**. Zrezygnowano również na tym etapie z automatycznej synchronizacji bankowej na rzecz manualnego wprowadzania danych oraz wykorzystania zaawansowanego generatora danych testowych (Mock Data).

---

2. Architektura Systemu

2.1. Backend (FastAPI + SQLAlchemy)

Serwer działa w oparciu o architekturę bezstanową (stateless) i standardy REST.

- **Wzorzec Bazy Danych:** Wykorzystano wzorzec **Joined Table Inheritance (JTI)**. Tabela główna `assets` przechowuje wspólne cechy aktywów, a podtypy (np. `savings_accounts`, `ike_accounts`, `crypto_wallets`) są połączone przez współdzielony klucz główny (PK/FK). Analogicznie zaimplementowano tabelę `holdings` dla inwestycji (z podtypami `bond_details`, `cfd_details`).
- **Walidacja i Schematy (Pydantic):** Zastosowano zaawansowane dyskryminowane unie (`Union` + `Literal`), co pozwala na dynamiczne mapowanie wariantów inwestycji (np. `StockCreate`, `BondCreate`) z jednego, uniwersalnego endpointu `/holdings/`.
- **Autentykacja:** Autorski moduł JWT. Bezpieczne hashowanie bcrypt, tokeny weryfikowane przez *dependency* `get_current_user`. Architektura zakłada łatwą rozbudowę o odświeżanie tokenów i blacklistę.

2.2. Frontend (Flutter + Riverpod)

- **Zarządzanie Stanem:** Aplikacja korzysta z nowoczesnego paradygmatu **Riverpod 2.0** (`AsyncNotifier`, `Notifier`), co gwarantuje pełną separację logiki biznesowej od interfejsu użytkownika (UI).
- **Komunikacja z API:** Scentralizowana klasa `ApiService` obsługująca żądania HTTP (GET, POST, DELETE) z automatycznym wstrzykiwaniem tokenów JWT do nagłówków.
- **UI & UX:** Wykorzystano bibliotekę `fl_chart` do interaktywnych wykresów alokacji oraz pakiet `intl` do profesjonalnego formatowania walutowego (np. `2,500,000.00 PLN`).

---

3. Zrealizowane Moduły Biznesowe

Zgodnie z wymogami projektu, zaimplementowano następujące sekcje:

A. Panel Główny (Dashboard)

Podsumowanie całego majątku użytkownika. Zamiast obciążać backend skomplikowanymi zapytaniami agregującymi, zaimplementowano podejście "The Flutter Way": `FinancialSummaryNotifier` nasłuchuje lokalnych stanów (`investmentsProvider`, `savingsProvider`, `expensesProvider`). Wartość netto (Total Wealth) wyliczana jest błyskawicznie po stronie klienta, zapewniając płynność interfejsu przy każdej zmianie (dodaniu/usunięciu aktywa) bez ponownego odpytywania API.

B. Oszczędności i Konta Emerytalne (Assets)

Zarządzanie standardowym kapitałem. Backend udostępnia rozdzielone "szufladki" dla poszczególnych produktów finansowych:

- `/assets/savings/` (Standardowe konta oszczędnościowe)
- `/assets/ppk/` (Pracownicze Plany Kapitałowe)
- `/assets/ike/` i `/assets/ikze/` (Konta z rocznymi limitami wpłat)

C. Inwestycje i Giełda (Holdings + Crypto)

Moduł inwestycyjny z "inteligentną zwrotnicą" API:

- **Giełda (Akcje, ETF, Obligacje, CFD):** Obsługiwana przez jeden potężny lejek `POST /holdings/`. Flutter automatycznie wstrzykuje wymagane parametry dla

instrumentów złożonych (np. data wykupu i emitent dla obligacji, dźwignia dla CFD), co niweluje błędy 422 Unprocessable Entity.
- **Kryptowaluty:** Stanowią osobną kategorię w architekturze bazy i posiadają własny endpoint `POST /assets/crypto/`. Po stronie frontendu łączą się w jeden widok portfela wraz z resztą aktywów.

D. Kalkulator Emerytalny (Retirement)

Oparty o regułę 4%. Moduł zrealizowany poprzez dwa *stateless* endpointy (`/retirement/zus-projection` oraz `/retirement/private-projection`) i zintegrowany z interfejsem graficznym. Zabezpiecza przed błędem dzielenia przez zero.

E. Wydatki (Transactions)

Moduł pozwalający na ręczne śledzenie kosztów życia, przypisywanie ich do kategorii i sygnalizowanie przekroczenia budżetu w Panelu Głównym (system alertów `dashboardAlertsProvider`).

---

4. Kluczowe Rozwiązania Inżynieryjne (Wdrożone z sukcesem)

Podczas integracji end-to-end napotkano i rozwiązano kilka kluczowych wyzwań architektonicznych:

1. **Kuloodporny tłumacz modeli (Frontend):** Tabele backendowe stosują różne konwencje nazewnictwa dla wartości kapitału (np. `current_balance`, `cash_balance`, `current_value`, `avg_purchase_price`). W `ApiService` zbudowano mechanizm, który iteruje po odpowiedzi z serwera, wyciąga wartość z dowolnego z tych kluczy i klonuje ją pod zunifikowany klucz `value`. Gwarantuje to poprawne mapowanie do flutterowego `AssetModel` w każdej sytuacji.
2. **Kaskadowe usuwanie osieroconych rekordów (Backend):** Rozwiązano problem z błędem `AssertionError` podczas usuwania obligacji i CFD. W głównym modelu (`models/holding.py`) wymuszono w SQLAlchemy parametry `cascade="all, delete-orphan"` przy relacjach. Zapewnia to integralność relacyjną przy akcji DELETE.
3. **Inteligentny Kosz (Frontend):** Dzięki dyskryminacji typów, funkcje usuwające w `ApiService` przyjmują zarówno `id` jak i `type`. Aplikacja dynamicznie układa prawidłowy adres URL zapytania DELETE na podstawie typu aktywa wyciągniętego z interfejsu (np. kieruje żądanie pod `/assets/crypto/{id}` dla Bitcoina, a pod `/holdings/{id}` dla Apple).

---

5. Przepływ Pracy (Git Workflow)

Zastosowano model wielogałęziowy (Branching Model) zbliżony do GitFlow:

- **Gałąź `main`:** Stabilna, działająca wersja aplikacji (chroniona).
- **Gałęzie `feature/*`:** Kod rozwijany równolegle przez programistów (np. `feature/backend-assets`, `feature/flutter-cards`).
- Aktualizacja gałęzi głównej następowała wyłącznie poprzez Pull Requesty po uprzednich testach środowiska lokalnego, co zapobiegło krytycznym konfliktom pomiędzy zespołem Backend (Dev A, Dev B) a Frontend (Dev C, Dev D).

---

6. Ograniczenia i Dalszy Rozwój (Next Steps)

Obecna wersja aplikacji z naddatkiem spełnia założenia dla wersji MVP (ZPI), jednak została architektonicznie przygotowana na natychmiastową rozbudowę:

1. **Brak operacji UPDATE (PUT):** Uzupełnienie endpointów oraz logiki UI do edytowania rekordów (np. zmiana ilości posiadanych akcji).
2. **Baza Danych PostgreSQL:** Obecnie wdrożono SQLite na potrzeby prototypu. Dzięki SQLAlchemy, migracja na relacyjną bazę PostgreSQL wymaga jedynie edycji w pliku `.env` (zmienna `DATABASE_URL`).
3. **Odświeżanie tokenów (Refresh Tokens):** Backend jest gotowy na wprowadzenie endpointu `/auth/refresh` oraz tabeli `refresh_tokens`, co przedłuży sesję użytkownika w sposób bezpieczny.
4. **Automatyzacja wycen (yfinance):** Dopisanie w backendzie "background task", który cyklicznie odpytuje zewnętrzne API i aktualizuje kolumnę `current_price` dla giełdowych tickerów z tabeli `holdings`.